

New Horizons · Centro de Capacitacion

Proyecto guiado: Tiendita Online

Paso a paso — rutas, parametros, guards, lazy loading y estado compartido con signals

Curso	Full Stack Java 25
Modulo / Sesion	Modulo 4 — Sesion 16: Routing, navegacion y manejo de estados
Docente	Edwin Barrientos
Duracion estimada	150–180 minutos
Nivel	Integrador (cubre las practicas 1, 2 y 3)
Entorno	Node 22 + Angular CLI 19 (standalone, sin NgModules)
Resultado	SPA de e-commerce con 12 rutas, 3 guards, 5 chunks lazy y un store con signals

1. Objetivos

Al terminar la guia el estudiante sera capaz de:

- Definir una tabla de rutas completa: rutas simples, con parametro, con `redirectTo` y comodin 404.
- Navegar sin recargar con `routerLink` y desde codigo con `router.navigate`.
- Leer parametros de ruta y *query params*, y usar la URL como fuente de verdad de los filtros.
- Proteger rutas con guards de entrada (`CanActivateFn`) y de salida (`CanDeactivateFn`).
- Cargar secciones bajo demanda con `loadComponent` y `loadChildren`.
- Centralizar el estado compartido en un store con `signal`, `computed` y `asReadonly()`.

2. Requisitos previos

- Node 22 o superior y Angular CLI 19 instalados.
- Haber visto la Sesion 16 (slides 5 a 21) o las practicas 1, 2 y 3.
- Un editor con soporte TypeScript (VS Code recomendado).

terminal

```
node -v      # v22.x
npm -v       # 10.x
npx ng version
```

3. Que vamos a construir

Una tienda online minima. Cada ruta existe para demostrar un tema del temario:

URL	Vista	Tema
/	Inicio	ruta vacia, pathMatch
/catalogo	Grilla de productos	query params
/catalogo/:id	Detalle del producto	parametro de ruta
/carrito	Carrito de compras	estado compartido
/login	Inicio de sesion	returnUrl
/checkout	Pago	lazy + 3 guards
/gracias/:pedidoId	Confirmacion	lazy + parametro
/admin	Panel (redirige)	loadChildren
/admin/productos	Editar precios	ruta hija
/admin/pedidos	Historial	ruta hija
/productos	Alias	redirectTo
cualquier otra	Pagina 404	comodin **

Regla de oro del router: Angular evalua las rutas **de arriba hacia abajo** y se queda con la primera que coincide. Por eso el comodin '*' va **siempre al final** del arreglo.

4. Desarrollo paso a paso

Paso 1 — Crear el proyecto

Sin routing automatico: la tabla de rutas la escribimos a mano para entenderla.

terminal

```
ng new tiendita-online --standalone --routing=false --style=css --ssr=false
cd tiendita-online
ng serve
```

Estructura de carpetas que vamos a crear (separar core de features evita el tipico app/ con 40 archivos sueltos):

estructura

```
src/app/
  core/      models/  services/  store/  guards/
  features/  inicio/  catalogo/  carrito/ checkout/  admin/  login/  no-encontrado/
```

Paso 2 — Modelo de datos

Primero los tipos: si el modelo esta claro, los componentes salen solos.

src/app/core/models/producto.model.ts

```
export type Categoria = 'polos' | 'tazas' | 'stickers';

export interface Producto {
  id: number; nombre: string; categoria: Categoria;
  precio: number; stock: number; emoji: string; descripcion: string;
}

export interface ItemCarrito { producto: Producto; cantidad: number; }

export interface Pedido {
  id: string; cliente: string; items: ItemCarrito[]; total: number; fecha: Date;
}
```

Paso 3 — Servicio de catalogo

El catalogo vive en un signal (no en un arreglo suelto) para que el panel de admin pueda modificarlo y la tienda se refresque sola. En una app real aca iría HttpClient.

src/app/core/services/producto.service.ts

```
@Injectable({ providedIn: 'root' })
export class ProductoService {
  private _productos = signal<Producto[]>([ /* 6 productos de ejemplo */ ]);

  productos = this._productos.asReadonly(); // lectura publica
  categorias = computed(() => // estado derivado
    [...new Set(this._productos().map(p => p.categoria))]);

  porId(id: number) { return this._productos().find(p => p.id === id); }

  actualizarPrecio(id: number, precio: number) {
    this._productos.update(l => l.map(p => p.id === id ? { ...p, precio } : p));
  }
}
```

Patron: el signal privado es el unico que se muta; afuera solo se expone asReadonly(). Asi nadie cambia el estado por la espalda del servicio.

Paso 4 — Store del carrito (estado compartido)

El corazon de la app. Todo lo que se puede calcular va en computed(): no se guarda el total, se deriva.

src/app/core/store/carrito.store.ts

```
@Injectable({ providedIn: 'root' })
export class CarritoStore {
  private _items = signal<ItemCarrito[]>([]);
  items = this._items.asReadonly();

  cantidad = computed(() => this._items().reduce((a, i) => a + i.cantidad, 0));
  total = computed(() => this._items()
    .reduce((a, i) => a + i.producto.precio * i.cantidad, 0));
  vacio = computed(() => this._items().length === 0); // lo usa un guard

  agregar(producto: Producto) {
    this._items.update(items => {
      const ex = items.find(i => i.producto.id === producto.id);
      return ex
        ? items.map(i => i.producto.id === producto.id
          ? { ...i, cantidad: i.cantidad + 1 } : i)
        : [...items, { producto, cantidad: 1 }];
    });
  }
}
```

```

    quitar(id: number) { this._items.update(i => i.filter(x => x.producto.id !== id)); }
    vaciar() { this._items.set([]); }
  }

```

Paso 5 — Store de sesion

Version minima para que el guard tenga algo que consultar. El token se guarda en `localStorage` para que sobreviva al F5.

`src/app/core/store/auth.store.ts`

```

@Injectable({ providedIn: 'root' })
export class AuthStore {
  private _usuario = signal<string | null>(localStorage.getItem('tiendita.usuario'));
  usuario = this._usuario.asReadonly();
  logueado = computed(() => this._usuario() !== null);

  login(usuario: string, clave: string): boolean {
    if (!usuario.trim() || !clave.trim()) return false;
    localStorage.setItem('tiendita.usuario', usuario);
    this._usuario.set(usuario);
    return true;
  }

  logout() { localStorage.removeItem('tiendita.usuario'); this._usuario.set(null); }
}

```

Paso 6 — Tabla de rutas y registro del router

Arrancamos con las rutas simples; los guards y el lazy loading los agregamos en los pasos 11 a 13.

`src/app/app.routes.ts`

```

export const routes: Routes = [
  { path: '', pathMatch: 'full', component: InicioComponent, title: 'Tiendita' },
  { path: 'catalogo', component: CatalogoComponent },
  { path: 'catalogo/:id', component: DetalleProductoComponent }, // parametro
  { path: 'carrito', component: CarritoComponent },
  { path: 'login', component: LoginComponent },
  { path: 'productos', redirectTo: 'catalogo', pathMatch: 'full' },
  { path: '**', component: NoEncontradoComponent }, // 404 al final
];

```

`src/app/app.config.ts`

```

export const appConfig: ApplicationConfig = {
  providers: [
    provideZoneChangeDetection({ eventCoalescing: true }),
    provideRouter(
      routes,
      withComponentInputBinding(), // params de ruta -> input() del componente
      withInMemoryScrolling({ scrollPositionRestoration: 'top' }),
    ),
  ],
};

```

`withComponentInputBinding()` es la parte que hace magia en el paso 9: el `:id` de la URL llega al componente como un `input()`, sin inyectar `ActivatedRoute`.

Paso 7 — Shell: nav con routerLink y router-outlet

Un solo router-outlet en la raiz. El header queda fuera: no se vuelve a renderizar al navegar, y ahí se nota que la SPA no recarga.

src/app/app.component.ts

```
@Component({
  selector: 'app-root',
  imports: [RouterOutlet, RouterLink, RouterLinkActive],
  template: `
    <header>
      <a routerLink="/" class="marca">Tiendita</a>
      <nav>
        <a routerLink="/" routerLinkActive="on"
          [routerLinkActiveOptions]="{ exact: true }">Inicio</a>
        <a routerLink="/catalogo" routerLinkActive="on">Catalogo</a>
        <a routerLink="/carrito" routerLinkActive="on">
          Carrito
          @if (carrito.cantidad() > 0) { <span class="badge">{{ carrito.cantidad() }}</span> }
        </a>
        <a routerLink="/admin" routerLinkActive="on">Admin</a>
      </nav>
    </header>

    <main><router-outlet /></main>
  `,
})
export class AppComponent {
  carrito = inject(CarritoStore); // el badge lee el store compartido
}
```

! Nunca usar href="/catalogo": recarga toda la app y se pierde el estado en memoria (adios carrito). Siempre routerLink.

Paso 8 — Catalogo con query params

Los filtros **no son estado del componente**: viven en la URL. Así el filtro es compatible, sobrevive al F5 y el botón atrás funciona.

src/app/features/catalogo/catalogo.component.ts

```
export class CatalogoComponent {
  productos = inject(ProductoService);
  private carrito = inject(CarritoStore);
  private route = inject(ActivatedRoute);
  private router = inject(Router);

  // query params leídos como signals
  categoria = toSignal(this.route.queryParamMap.pipe(map(q => q.get('categoria'))),
    { initialValue: null });
  orden = toSignal(this.route.queryParamMap.pipe(map(q => q.get('orden'))),
    { initialValue: null });

  // la lista visible se CALCULA a partir de la URL + el catalogo
  visibles = computed(() => {
    let lista = this.productos.productos();
    if (this.categoria()) lista = lista.filter(p => p.categoria === this.categoria());
    if (this.orden() === 'precio') lista = [...lista].sort((a, b) => a.precio - b.precio);
    return lista;
  });

  filtrar(categoria: string | null) {
    this.router.navigate([], {
      relativeTo: this.route,
      queryParams: { categoria },
      queryParamsHandling: 'merge', // conserva los otros params
    });
  }
}
```

```
}  
}
```

Paso 9 — Detalle con parametro de ruta

Gracias a `withComponentInputBinding()`, el `:id` llega como `input()`. Llega como *string*, de ahí el `transform`.

`src/app/features/catalogo/detalle-producto.component.ts`

```
export class DetalleProductoComponent {  
  id = input.required({ transform: (v: string) => Number(v) });  
  
  private productos = inject(ProductoService);  
  private carrito = inject(CarritoStore);  
  private router = inject(Router);  
  
  producto = computed(() => this.productos.porId(this.id()));  
  
  comprarYa() { // navegacion programatica  
    const p = this.producto();  
    if (p) { this.carrito.agregar(p); this.router.navigate(['/carrito']); }  
  }  
}
```

La plantilla debe manejar el caso `producto()` indefinido (`/catalogo/999`). Alternativa clásica del slide 9: `toSignal(route.paramMap.pipe(map(p => p.get('id'))))`. Vale la pena mostrar ambas.

Paso 10 — Carrito: la misma instancia del store

`src/app/features/carrito/carrito.component.ts`

```
export class CarritoComponent {  
  store = inject(CarritoStore); // MISMA instancia que el header y el catalogo  
  private router = inject(Router);  
  irAlCheckout() { this.router.navigate(['/checkout']); }  
}
```

Prueba de que el estado está centralizado: agrega desde el catálogo y el badge del header sube sin que exista ningún `@Output` ni servicio de eventos. `providedIn: 'root'` = una sola instancia para toda la app.

Paso 11 — Guards: proteger la entrada y la salida

Tres guards. Los dos primeros son de entrada y se evalúan **en orden**; el tercero controla la salida.

`src/app/core/guards/auth.guard.ts`

```
export const authGuard: CanActivateFn = (_ruta, estado) => {  
  const auth = inject(AuthStore);  
  const router = inject(Router);  
  if (auth.logueado()) return true;  
  // guarda de donde venia para volver despues del login  
  return router.createUrlTree(['/login'], { queryParams: { returnUrl: estado.url } });  
};
```

src/app/core/guards/carrito-no-vacio.guard.ts

```
export const carritoNoVacioGuard: CanActivateFn = () => {
  const carrito = inject(CarritoStore);
  const router = inject(Router);
  return carrito.vacio()
    ? router.createUrlTree(['/catalogo'], { queryParams: { aviso: 'carrito-vacio' } })
    : true;
};
```

src/app/core/guards/salir-confirmado.guard.ts

```
export interface PuedeSalir { puedeSalir(): boolean; }

export const salirConfirmadoGuard: CanDeactivateFn<PuedeSalir> = (componente) => {
  if (componente.puedeSalir()) return true;
  return confirm('Tienes datos sin confirmar. Salir de todas formas?');
};
```

En el login, leer el returnUrl y volver ahí:

src/app/features/login/login.component.ts

```
returnUrl = toSignal(this.route.queryParamMap.pipe(map(q => q.get('returnUrl'))),
  { initialValue: null });

entrar() {
  if (!this.auth.login(this.usuario, this.clave)) { this.error.set(true); return; }
  this.router.navigateByUrl(this.returnUrl() ?? '/');
}
```

Paso 12 — Checkout con lazy loading

El componente se descarga solo cuando se visita la ruta. Se agrega a la tabla de rutas con los tres guards:

src/app/app.routes.ts (agregar)

```
{
  path: 'checkout',
  canActivate: [authGuard, carritoNoVacioGuard], // se evalúan en orden
  canDeactivate: [salirConfirmadoGuard],
  loadComponent: () => import('./features/checkout/checkout.component')
    .then(m => m.CheckoutComponent),
},
{
  path: 'gracias/:pedidoId', // lazy + parametro
  loadComponent: () => import('./features/checkout/gracias.component')
    .then(m => m.GraciasComponent),
},
```

El componente implementa PuedeSalir para dialogar con el guard de salida:

src/app/features/checkout/checkout.component.ts

```
export class CheckoutComponent implements PuedeSalir {
  carrito = inject(CarritoStore);
  private router = inject(Router);

  cliente = '';
  private tocado = signal(false); // estado LOCAL del componente
  private confirmado = signal(false);

  puedeSalir() { return this.confirmado() || !this.tocado(); }

  confirmar() {
    const pedido = this.carrito.confirmar(this.cliente); // descuenta stock y vacía
    this.confirmado.set(true);
    this.router.navigate(['/gracias', pedido.id]);
  }
}
```

Paso 13 — Area admin: loadChildren y rutas hijas

Un area completa cargada bajo demanda, con su propio router-outlet anidado.

src/app/features/admin/admin.routes.ts

```
export const ADMIN_ROUTES: Routes = [
  {
    path: '',
    component: AdminLayoutComponent, // layout con router-outlet propio
    children: [
      { path: '', redirectTo: 'productos', pathMatch: 'full' },
      { path: 'productos', loadComponent: () =>
        import('./admin-productos.component').then(m => m.AdminProductosComponent) },
      { path: 'pedidos', loadComponent: () =>
        import('./admin-pedidos.component').then(m => m.AdminPedidosComponent) },
    ],
  },
];
```

src/app/features/admin/admin-layout.component.ts

```
@Component({
  imports: [RouterOutlet, RouterLink, RouterLinkActive],
  template: `
    <nav class="subnav">
      <a routerLink="productos" routerLinkActive="on">Productos</a> <!-- relativa -->
      <a routerLink="pedidos" routerLinkActive="on">Pedidos</a>
    </nav>
    <router-outlet /> <!-- outlet ANIDADO: aca van las rutas hijas -->
  `,
})
export class AdminLayoutComponent {}
```

src/app/app.routes.ts (agregar)

```
{
  path: 'admin',
  canActivate: [AuthGuard],
  loadChildren: () => import('./features/admin/admin.routes')
    .then(m => m.ADMIN_ROUTES),
},
```

Paso 14 — 404 y redirects

src/app/features/no-encontrado/no-encontrado.component.ts

```
@Component({
  imports: [RouterLink],
  template: `
    <h1>404</h1>
    <p>La ruta <code>{{ url }}</code> no existe.</p>
    <a routerLink="/" class="btn">Volver al inicio</a>
  `,
})
export class NoEncontradoComponent {
  url = inject(Router).url;
}
```

Revisar que el arreglo de rutas termine así, en este orden exacto:

orden final de app.routes.ts

```
'', 'catalogo', 'catalogo/:id', 'carrito', 'login',
'checkout', 'gracias/:pedidoId', 'admin', 'productos' (redirect), ''
```

Paso 15 — Compilar y verificar el lazy loading

terminal

```
npm run build
```

En la salida deben aparecer los chunks separados. Si el checkout no figura como chunk propio, el loadComponent no quedo bien:

salida esperada de ng build

Lazy chunk files	Names	Raw size
chunk-XXXXXXX.js	checkout-component	2.84 kB
chunk-XXXXXXX.js	admin-productos-component	1.99 kB
chunk-XXXXXXX.js	admin-pedidos-component	1.85 kB
chunk-XXXXXXX.js	admin-routes	1.60 kB
chunk-XXXXXXX.js	gracias-component	1.50 kB

5. Checklist de verificacion

Probar en el navegador, en este orden. La columna *Tema* es lo que hay que poder explicar.

OK	Prueba	Resultado esperado
✓	Click en Catalogo desde el nav	no recarga (Network sin documento)
✓	Boton “ver solo tazas”	/catalogo?categoria=tazas &orden=precio
✓	Copiar la URL filtrada en otra pestana	mismo filtro; F5 y atras funcionan
✓	Click en un producto	/catalogo/3 con su detalle
✓	Abrir /catalogo/999	mensaje ``producto no encontrado''
✓	Agregar 2 productos	el badge del header sube a 2
✓	Ir a /admin sin sesion	/login?returnUrl=%2Fadmin
✓	Loguearse	vuelve solo a /admin/productos
✓	Abrir /checkout con carrito vacio	/catalogo?aviso=carrito-vacio
✓	Con items, ir a pagar	Network descarga el chunk del checkout
✓	Escribir el nombre y salir	aparece el confirm del canDeactivate
✓	Confirmar pedido	/gracias/P-XXXXXX y el badge desaparece
✓	Admin: cambiar un precio	el catalogo ya muestra el precio nuevo
✓	Escribir /cualquier-cosa	pagina 404

Comportamiento esperado, no un bug: el carrito vive en memoria, así que F5 lo vacía. La sesión, en cambio, está en `localStorage` y sobrevive. El reto 3 resuelve la diferencia.

6. Retos adicionales

1. Ruta `/ofertas` que muestre solo productos de menos de S/ 35.
2. Probar `routerLinkActiveOptions` para que el link *Catalogo* también quede activo estando en `/catalogo/3`.
3. Persistir el carrito en `localStorage` con un `effect()` en el store y rehidratarlo al iniciar.
4. Agregar un campo `rol` en `AuthStore` y un `adminGuard` que solo deje entrar a `/admin` si el rol es `admin`.
5. Nuevo `computed` en el store: descuento del 10% cuando el total pasa de S/ 100, mostrado en carrito y checkout.
6. Usar un `resolve` en `catalogo/:id` para redirigir al 404 cuando el producto no existe.
7. Activar `withPreloading(PreloadAllModules)` y comparar en Network cuando se descargan los chunks.

7. Entregables

- Proyecto que compila con `ng build` sin errores y con al menos 3 chunks lazy.
- Captura de la pestaña Network mostrando el chunk del checkout al navegar a `/checkout`.
- Captura de la redirección del guard: `/login?returnUrl=...`
- Captura del badge del header con el carrito cargado, junto al catalogo.
- Respuesta escrita, en dos o tres líneas cada una:
 - Por que los filtros del catalogo estan en la URL y no en un `signal` del componente.
 - Que diferencia hay entre `loadComponent` y `loadChildren`.
 - Por que el total del carrito es un `computed` y no un `signal`.